

Sistema de Tickets IT

Documentación Técnica para Auditoría

Tecnologías Utilizadas:

Backend: PHP 7.x con PDO MySQL

Frontend: HTML5, CSS3 (Bootstrap 5), JavaScript (Vanilla)

Base de Datos: MySQL/MariaDB

Dependencias: PhpSpreadsheet, Firebase JWT, Google Auth

APIs Externas: Firebase Cloud Messaging (FCM)

Fecha de Generación: 17 de diciembre de 2025

Índice

1. Alcance del Documento

Esta documentación técnica abarca la arquitectura del sistema de gestión de tickets de soporte IT. Cubre los componentes backend desarrollados en PHP, las vistas frontend HTML, componentes JavaScript integrados y estilos CSS aplicados. Se incluye el módulo de API para procesamiento de solicitudes, gestión de base de datos, autenticación de usuarios y generación de reportes. Se excluyen configuraciones de servidor, variables de entorno sensibles y documentación de deployment.

2. Descripción General del Sistema

El sistema constituye una plataforma de gestión de solicitudes de soporte técnico operada bajo arquitectura cliente-servidor. Implementa una experiencia de usuario segmentada con portal público para creación de tickets, dashboard autenticado para usuarios solicitantes y panel administrativo para personal IT. El flujo general permite que usuarios registren incidentes sin autenticación mediante número de ficha, mientras que el personal IT accede a un panel administrativo para gestión, asignación y resolución de tickets. La capa de API proporciona endpoints JSON para operaciones CRUD sobre tickets y estadísticas.

3. Estructura del Proyecto

Listing 1: Distribución de carpetas

```
/
    config/
        database.php
    api/
        tickets.php
        tickets_admin.php
        tickets_publico.php
        reportes.php
    index.php
    login.php
    dashboard.php
    admin.php
    logout.php
    crear_usuarios.php
    api.php
```

```
composer.json
vendor/
```

El directorio `config` centraliza la configuración de base de datos. El directorio `api` contiene endpoints JSON especializados. Los archivos PHP en raíz implementan vistas y lógica de presentación. El directorio `vendor` aloja dependencias Composer.

4. Componentes Backend (PHP)

4.1. `config/database.php`

Implementa la clase `Database` para gestión centralizada de conexiones PDO a MySQL. Proporciona funciones globales de utilidad para manejo de sesiones, validación de roles, formateo de fechas y codificación cromática de prioridades y estados.

```
public function getConnection() {
    $this->conn = new PDO("mysql:host=" . $this->host .
        ";dbname=" . $this->db_name ,
        $this->username , $this->password);
    $this->conn->setAttribute(PDO::ATTR_ERRMODE ,
        PDO::ERRMODE_EXCEPTION);
}
```

Listing 2: Clase Database

4.2. `index.php`

Proporciona la vista pública del sistema accesible sin autenticación. Implementa interfaz de búsqueda de tickets mediante número de ficha y nombre de solicitante. Contiene formulario para creación de nuevos tickets con campos de categoría, prioridad y descripción. Integra estilos Bootstrap 5 y gradientes CSS personalizados.

```
<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-12 col-lg-10">
            <div class="card main-card">
```

Listing 3: Estructura portal público

4.3. `login.php`

Maneja autenticación de usuarios IT mediante email y contraseña. Valida credenciales contra tabla usuarios utilizando `password_verify()`. Establece variables de sesión post-

login incluyendo ID, nombre, email y rol. Implementa redirección condicional a dashboard según rol de usuario.

```
if (password_verify($password, $row['password'])) {  
    $_SESSION['usuario_id'] = $row['id'];  
    $_SESSION['rol'] = $row['rol'];  
}
```

Listing 4: Validación de credenciales

4.4. dashboard.php

Proporciona interfaz personalizada para usuarios autenticados con rol solicitante. Muestra lista de tickets del usuario, estadísticas y opción de crear nuevos tickets. Implementa navegación basada en rol con secciones condicionales para personal IT. Contiene modales para visualización de detalles y timeline de eventos.

```
<?php if ($_SESSION['rol'] === 'it'): ?>  
    <li class="nav-item">  
        <!-- opciones especificas IT -->  
    </li>  
<?php endif; ?>
```

Listing 5: Navbar condicional

4.5. admin.php

Panel de administración exclusivo para personal IT con rol verificado. Gestiona visualización de todos los tickets, asignación de recursos, resolución y cierre de incidentes. Implementa autenticación de administrador separada en sesión. Proporciona estadísticas globales y listado de usuarios IT disponibles.

```
if (!isset($_SESSION['admin_id'])) {  
    $error_message = 'Usuario no tiene permisos';  
}
```

Listing 6: Verificación admin

4.6. logout.php

Destruye sesión de usuario y elimina cookies asociadas. Vacía array de sesión global y ejecuta función `session_destroy()`. Redirige a página de login post-destrucción de sesión.

```
$_SESSION = array();
session_destroy();
header("Location: login.php");
```

Listing 7: Destrucción de sesión

4.7. crear_usuarios.php

Script de inicialización que inserta usuarios IT predefinidos en tabla usuarios. Hash de contraseñas utilizando PASSWORD_DEFAULT. Asigna rol 'it' y estado 'disponible' a usuarios creados.

```
$passwordHash = password_hash($usuario['password'],
    PASSWORD_DEFAULT);
$stmt->bindParam(1, $usuario['nombre']);
```

Listing 8: Creación de usuarios

4.8. api.php

Router principal de API que implementa endpoints JSON con CORS habilitado. Gestiona conexión a base de datos centralizada, logging en archivo api_debug.log, configuración de Firebase Cloud Messaging y gestión de tokens FCM. Proporciona funciones para envío de notificaciones push a clientes registrados.

```
header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, OPTIONS');
```

Listing 9: Configuración CORS

5. Componentes Backend - APIs Especializadas

5.1. api/tickets.php

Endpoint JSON para operaciones CRUD sobre tickets con validación de sesión autenticada. Proporciona acciones: obtener tickets del usuario, obtener estadísticas personalizadas, obtener personal IT disponible, crear tickets, tomar tickets en asignación. Diferencia permisos según rol: solicitante visualiza solo sus propios tickets, personal IT visualiza todos.

```
if ($_SESSION['rol'] === 'solicitante') {
    $query .= "WHERE t.solicitante_id = ?";
}
```

```
$stmt->bindParam(1, $_SESSION['usuario_id']);
```

Listing 10: Obtención de tickets por rol

5.2. api/tickets_admin.php

Endpoint especializado para panel administrativo con validación de sesión admin. Implementa operaciones: obtención de tickets recientes, obtención de todos los tickets, obtención de estadísticas globales, tomar ticket en asignación, resolver ticket. Requiere header admin_id en sesión.

```
if (!isset($_SESSION['admin_id'])) {
    echo json_encode(['success' => false,
        'message' => 'No autorizado']);
}
```

Listing 11: Verificación admin

5.3. api/tickets_publico.php

Endpoint sin autenticación para portal público. Implementa búsqueda de tickets mediante número de ficha y nombre de solicitante. Proporciona creación de tickets públicos con validación de campos obligatorios. Gestiona archivos adjuntos y cierre de tickets con captura de satisfacción.

```
$query = "SELECT t.* FROM tickets t
        WHERE t.numero_ficha = ?
        AND t.solicitante_nombre = ?";
```

Listing 12: Búsqueda pública

5.4. api/reportes.php

Módulo de generación de reportes en formato XLSX utilizando PhpSpreadsheet. Exporta datos de tickets con filtros por estado, fecha y tipo. Implementa validación de permisos admin previo a descarga. Genera archivo con estructura de columnas: ID, número ficha, solicitante, título, descripción, prioridad, categoría, estado, asignación, fechas y resolución.

```
use PhpOffice\PhpSpreadsheet\Spreadsheet;
use PhpOffice\PhpSpreadsheet\Writer\Xlsx;
$spreadsheet = new Spreadsheet();
$writer = new Xlsx($spreadsheet);
```

Listing 13: Generación Excel

6. Componentes Frontend - Vistas HTML

6.1. index.php (Sección HTML)

Define estructura HTML5 con viewport responsivo. Implementa layout CSS con gradientes Bootstrap 5. Contiene secciones tabuladas para búsqueda de tickets y creación de incidentes. Utiliza iconografía FontAwesome 6.0 integrada en elementos de interfaz. Incluye validación HTML5 en formularios.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/
    bootstrap/5.3.0/css/bootstrap.min.css">
```

Listing 14: Estructura HTML

6.2. login.php (Sección HTML)

Define formulario de autenticación con estructura tarjeta centrada. Implementa inputs con iconografía FontAwesome integrada. Utiliza clases Bootstrap 5 para validación y responsividad. Incluye asistencia visual para credenciales de prueba.

```
<form id="loginForm">
  <div class="input-group">
    <span class="input-group-text">
      <i class="fas fa-envelope"></i>
    </span>
    <input type="email" class="form-control">
  </div>
</form>
```

Listing 15: Formulario login

6.3. dashboard.php (Sección HTML)

Implementa layout de dos columnas con navbar responsiva. Contiene navegación principal con opciones condicionales por rol. Incluye secciones dinámicas para lista de tickets, estadísticas y formularios. Utiliza modales Bootstrap para visualización de detalles. Implementa timeline CSS personalizado para historial de eventos.

```
<nav class="navbar navbar-expand-lg navbar-dark
```



```

        navbar-custom">
        <a class="navbar-brand">
            <i class="fas fa-headset me-2"></i>
        </a>
    </nav>

```

Listing 16: Navbar dashboard

6.4. admin.php (Sección HTML)

Define panel administrativo con navegación especializada para roles IT. Implementa tablas de datos para visualización de tickets. Contiene modales para edición y asignación. Utiliza tarjetas estadísticas con gradientes. Incluye timeline visual de eventos de tickets.

```

<div class="card card-dashboard">
    <div class="card-body">
        <h5 class="card-title">Gest i n de Tickets</h5>
    </div>
</div>

```

Listing 17: Panel admin

7. Componentes JavaScript

7.1. Validación de Formularios (index.php)

Script integrado que implementa validación de campos obligatorios y envío asincrónico mediante Fetch API. Valida número de ficha, nombre y descripción antes de POST. Maneja respuestas JSON con mensajes de error personalizados. Implementa indicador de carga visual durante procesamiento.

```

document.getElementById('loginForm')
    .addEventListener('submit', function(e) {
        e.preventDefault();
        const formData = new FormData(this);
        fetch('login.php', {
            method: 'POST',
            body: formData
        })
    })

```

Listing 18: Validación form

7.2. Gestión de Sesión (login.php)

Script de autenticación que consume endpoint login.php mediante POST asincrónico. Valida respuesta JSON para éxito o error. Redirige a dashboard post-autenticación exitosa. Muestra mensajes de alerta contextualizados para usuario.

```
.then(data => {
  if (data.success) {
    window.location.href = 'dashboard.php';
  } else {
    mensajeDiv.innerHTML =
      '<div class="alert alert-danger">'
      + data.message + '</div>';
  }
})
```

Listing 19: Lógica login

7.3. Interactividad de Navegación (dashboard.php)

Scripts para activación/desactivación de secciones dinámicas mediante toggle de clase CSS. Implementa carga asincrónica de datos de APIs especializadas. Maneja eventos click en elementos de navegación. Actualiza contenido sin recarga de página.

```
function mostrarSeccion(seccion) {
  document.querySelectorAll('.section')
    .forEach(el => el.classList.remove('active'));
  document.getElementById(seccion)
    .classList.add('active');
}
```

Listing 20: Toggle secciones

8. Componentes de Estilo (CSS)

8.1. Estilos Globales (index.php)

Define variables CSS personalizadas para colores corporativos (primario: #667eea, secundario: #764ba2). Implementa gradientes lineales en elemento body. Define espaciado y bordes redondeados estándar. Estilos aplicados globalmente a controles de formulario (input, select, button).

Listing 21: Variables CSS

```
:root {
```

```

--primary-color: #667eea;
--secondary-color: #764ba2;
--success-color: #28a745;
--warning-color: #ffc107;
--danger-color: #dc3545;
}

```

8.2. Estilos de Componentes (dashboard.php)

Define clases para tarjetas de ticket con efectos hover. Implementa indicadores visuales de estado: abierto (azul), en proceso (amarillo), resuelto (verde), cerrado (gris). Estilos para timeline visual con línea vertical y marcadores. Proporciona clases de prioridad con bordes coloreados izquierdos.

Listing 22: Estilos prioridad

```

.ticket-priority-urgente {
    border-left: 5px solid var(--danger-color);
}
.ticket-priority-media {
    border-left: 5px solid var(--primary-color);
}

```

8.3. Estilos de Navegación (dashboard.php)

Define navbar con gradiente de colores corporativos. Implementa box-shadow para profundidad. Estilos para elementos collapse responsivos. Colores de texto contrastantes para accesibilidad. Media queries para adaptación en dispositivos móviles.

Listing 23: Navbar custom

```

.navbar-custom {
    background: linear-gradient(135deg,
        var(--primary-color),
        var(--secondary-color));
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

```

8.4. Estilos Modal (admin.php)

Define encabezado de modal con gradiente corporativo. Implementa colores de texto diferenciados. Define espaciado interno consistente. Estilos para botones dentro de modal. Media queries para responsividad.

Listing 24: Modal header

```
.modal-header-custom {
    background: linear-gradient(135deg,
        var(--primary-color),
        var(--secondary-color));
    color: white;
}
```

9. Configuración de Dependencias

El archivo `composer.json` especifica tres dependencias principales: `phpoffice/phpspreadsheet` versión 4.3 para generación de reportes en formato XLSX, `firebase/php-jwt` versión 6.11 para manejo de tokens JWT, y `google/auth` versión 1.47 para integración con servicios Google. Estas dependencias se ubican en directorio `vendor` post-instalación.

Listing 25: Dependencias Composer

```
{
    "require": {
        "phpoffice/phpspreadsheet": "^4.3",
        "firebase/php-jwt": "^6.11",
        "google/auth": "^1.47"
    }
}
```

10. Integración de APIs Externas

10.1. Firebase Cloud Messaging (FCM)

El sistema integra FCM para envío de notificaciones push. Requiere archivo `service-account-key.json` con credenciales de proyecto Firebase. Implementa generación de JWT personalizado para obtención de access tokens. Gestiona tokens FCM en tabla de base de datos dedicada. Proporciona función `get_fcm_tokens()` para recuperación de dispositivos registrados.

```
$header = json_encode(['alg' => 'RS256',
    'typ' => 'JWT']);
$payload = json_encode([
    'iss' => $service_account['client_email'],
    'scope' => 'https://www.googleapis.com/auth/
        firebase.messaging'
```

```
]);
```

Listing 26: Generación JWT

11. Autenticación y Autorización

El sistema implementa dos flujos de autenticación: portal público sin sesión para creación de tickets mediante número de ficha, y dashboard autenticado con validación de credenciales contra tabla usuarios. Verifica contraseña con `password_verify()` contra hash almacenado. Asigna rol en sesión (solicitante o it). Panel admin requiere rol específico 'it' con sesión separada. Funciones de utilidad validan presencia de sesión activa en vistas protegidas.

12. Gestión de Base de Datos

La capa de base de datos utiliza PDO MySQL con charset utf8mb4. Implementa prepared statements para prevención de inyección SQL. La clase `Database` centraliza configuración de conexión. Manejo de transacciones en operaciones críticas de actualización. Métodos de error manejados con excepciones `PDOException`. Tabla principal tickets almacena incidentes con campos: id, numero_ficha, solicitante_nombre, titulo, descripcion, prioridad, categoria, estado, asignado_a, fechas de creación/resolución/cierre, satisfaccion, resolucion.

13. Validación de Datos

El sistema implementa validación en múltiples capas. Frontend valida mediante atributos HTML5 (required, type, pattern). Backend valida mediante verificación de existencia de parámetros POST/GET. API endpoints validan tipos de datos antes de procesamiento. Prepared statements previenen inyección SQL. Archivos adjuntos validados contra lista blanca de extensiones permitidas.

14. Logging y Auditoría

El archivo `api.php` implementa logging en tiempo real escribiendo en archivo `api_debug.log`. Registra eventos de conexión a base de datos, generación de tokens, envío de notificaciones. La función `debug_log()` agrega timestamp a cada evento. Redirecciona logs simultáneamente a archivo y `error_log` de PHP. Permite trazabilidad de operaciones de API para debugging.

15. Manejo de Archivos

El sistema soporta carga de archivos adjuntos a tickets. Define límite de tamaño máximo 100MB. Implementa lista blanca de tipos MIME permitidos: imágenes (jpg, png, webp), documentos (pdf, docx, xlsx), videos (mp4, webm). Almacena archivos en directorio `uploads/tickets/`. Registra metadata de archivos en tabla `ticket_attachments` incluyendo nombre original, tamaño y tipo MIME.

```
'allowed_types' => [  
  'jpg', 'jpeg', 'png', 'gif', 'webp',  
  'pdf', 'doc', 'docx', 'xls', 'xlsx'  
]
```

Listing 27: Validación archivos